Service Overview

ภาพรวมของบริการ (Service Overview)

RecommendRescueTeam Service

0. Github repo

https://github.com/kamonphankanthayod/recommend-rescue-team-service

1. Service Owner

นายกมลพันธ์ กันธายอด รหัสนักศึกษา 6609520116 ภาคปกติ

2. Service Purpose

RecommendRescueTeam Service เป็นบริการแบบ Microservice ที่ทำหน้าที่เป็น Decision-Support Engine สำหรับแนะนำทีมกู้ภัยที่เหมาะสมที่สุดต่อคำร้องขอความช่วยเหลือ (Rescue Request) ภายใต้บริบทของเหตุการณ์ภัยพิบัติ (Incident)

บริการนี้มีหน้าที่: วิเคราะห์ข้อมูลคำร้องขอความช่วยเหลือ, ประเมินความเหมาะสมของทีมกู้ภัยตามเกณฑ์ที่กำหนด, จัดลำดับทีมตามคะแนนความเหมาะสม, สร้างคำแนะนำที่สามารถอธิบายเหตุผลได้ (Explainable Recommendation), คำนวณค่าความเชื่อมั่น (Confidence Score), รองรับการประเมินใหม่เมื่อสถานการณ์เปลี่ยนแปลง

บริการนี้ ไม่ได้ทำหน้าที่ dispatch ทีมกู้ภัย แต่ทำหน้าที่เป็นกลไกสนับสนุนการตัดสินใจให้กับ Dispatcher และ Manage Dispatch Service

3. Pain Point ที่แก้ไข

ในสถานการณ์ภัยพิบัติจริง Dispatcher ต้องตัดสินใจอย่างรวดเร็วภายใต้ข้อจำกัดด้านเวลา ความรุนแรงของเหตุการณ์ และทรัพยากรที่จำกัด การตัดสินใจโดยอาศัยการประเมินด้วยมนุษย์เพียงอย่างเดียวอาจก่อให้เกิดปัญหา เช่น เลือกทีมที่ไม่มีความเชี่ยวชาญตรงกับประเภทเหตุการณ์, ส่งทีมที่อยู่ไกลเกินไป ทำให้เกิดความล่าช้า, ใช้ทรัพยากรไม่เหมาะสมกับระดับความรุนแรง, การตัดสินใจไม่สม่ำเสมอในเหตุการณ์ลักษณะคล้ายกัน, ไม่สามารถปรับตัวทันเมื่อสถานการณ์เปลี่ยนแบบ real-time

บริการนี้จึงช่วยทำให้กระบวนการประเมินเป็นระบบ, ลด Human Error, เพิ่มความโปร่งใส (Explainability), รองรับการปรับประเมินใหม่เมื่อข้อมูลเปลี่ยนแปลง

4. Target Users

- Dispatcher
- Manage Dispatch Service
- บริการนี้ไม่ได้ออกแบบให้ผู้ประสบภัยเรียกใช้โดยตรง แต่เป็น backend decision-support service สำหรับ Dispatcher และระบบอื่นใน Disaster Response Ecosystem

5. Service Boundary

- In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ)
    - วิเคราะห์ Rescue Request ร่วมกับบริบทของ Incident

    - คัดกรองทีมกู้ภัยที่ผ่านเงื่อนไขขั้นต่ำ (Candidate Filtering)
    - คำนวณคะแนนความเหมาะสม (Scoring Mechanism)
    - จัดลำดับทีม (Ranking)
    - สร้าง Recommendation Record
    - คำนวณ Confidence Score
    - จัดการ Lifecycle ของ Recommendation (PENDING / CALCULATING / GENERATED / ACCEPTED / REJECTED / SUPERSEDED / FAILED / EXPIRED)
    - เก็บ Score Breakdown และ Explanation เพื่อรองรับ Audit
    - รองรับ Re-evaluation แบบ Asynchronous
    - จัดเก็บ Model/Rule Version Metadata

- Out-of-scope / Not Responsible For (ไม่รับผิดชอบ)
    - การจัดการข้อมูล Incident Master Data
    - การสร้างหรือจัดการ Rescue Request
    - การจัดลำดับความสำคัญของคำร้อง (เป็นหน้าที่ Rescue Prioritization Service)
    - การ dispatch ทีมกู้ภัย
    - การจัดการความพร้อมใช้งานของทีมโดยตรง
    - การวางแผนเส้นทางหรือการจัดการโลจิสติกส์

6. Autonomy / Decision Logic

บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:

- การกำหนดเกณฑ์การคัดกรองทีมกู้ภัย
- การคำนวณคะแนนความเหมาะสมของแต่ละทีม
- การจัดลำดับทีม
- การคำนวณ Confidence Score ของผลลัพธ์
- การกำหนดสถานะของ Recommendation
- การตัดสินใจ Re-evaluate เมื่อบริบทเปลี่ยน

แนวคิดการคำนวณคะแนน:

Suitability Score อาจคำนวณจาก

- ความตรงกันของความเชี่ยวชาญกับประเภทเหตุการณ์
- ระยะทางจากจุดเกิดเหตุ
- ความพร้อมใช้งานของทีม
- ระดับความรุนแรงของเหตุการณ์
- เวลาตอบสนองโดยประมาณ

คะแนนรวมจะถูกเก็บพร้อมรายละเอียดการคำนวณ (Score Breakdown) เช่น:

- specialization_score
- distance_score
- availability_score
- severity_weight

Explainability

ทุก Recommendation จะเก็บ Explanation Metadata เพื่อให้ Dispatcher เข้าใจเหตุผลของการจัดอันดับ

Re-evaluation

รองรับการประเมินซ้ำแบบบังคับ (Force Re-evaluate) โดยระบบจะทำการเปลี่ยนสถานะคำแนะนำเดิมเป็น SUPERSEDED เพื่อรักษาประวัติการตัดสินใจ (Audit Trail) และสร้างคำแนะนำชุดใหม่ขึ้นมาแทน

ระบบรองรับการประเมินใหม่เมื่อ:

- ความพร้อมใช้งานของทีมเปลี่ยน
- ระดับความรุนแรงของเหตุการณ์เปลี่ยน
- รายละเอียดคำร้องเปลี่ยน

มีการกำหนด Cooldown Window และ Limit ต่อจำนวนครั้งการประเมินใหม่ เพื่อป้องกัน Re-evaluation Storm

บริการสามารถตัดสินใจสร้างและจัดลำดับคำแนะนำได้เองภายใต้ business rules ที่กำหนด โดยไม่ต้องรอการอนุมัติจากมนุษย์ในขั้นตอนการประเมินปกติ ทั้งนี้ Dispatcher ยังคงเป็นผู้ตัดสินใจขั้นสุดท้ายในการเลือกทีมเพื่อ dispatch

7. Owned Data

- Recommendation Records
  ข้อมูลผลลัพธ์การแนะนำทีมกู้ภัย ที่ถูกสร้างขึ้นจากกระบวนการประเมินในแต่ละครั้ง โดยข้อมูลดังกล่าวสะท้อนผลการตัดสินใจ ณ เวลาหนึ่ง และใช้สำหรับแสดงผลแก่ Dispatcher, การติดตามสถานะของคำแนะนำ, การตรวจสอบย้อนหลัง, การรองรับ Re-evaluation
  เป็น Immutable Snapshot: ข้อมูลนี้เมื่อ GENERATED แล้วจะไม่ถูกเปลี่ยนแปลงแก้ไขคะแนนย้อนหลัง เพื่อใช้เป็นหลักฐาน (Audit) ว่าทำไมถึงแนะนำทีมนี้ ณ เวลานั้น

- Ranked Team Snapshot
  ทุกครั้งที่มีการสร้างคำแนะนำ ระบบจะเก็บ Snapshot ของผลการจัดลำดับทีม เพื่อป้องกันการคำนวณซ้ำและรองรับ Explainability ข้อมูลชุดนี้ประกอบด้วยรายชื่อทีมที่ผ่านการคัดกรอง, ลำดับ (Rank), คะแนนรวม, รายละเอียดคะแนนย่อย (Score Breakdown), คำอธิบายเหตุผลของการให้คะแนน (Explanation Metadata)
  ข้อมูลนี้ถือเป็น Decision Snapshot และจะไม่ถูกแก้ไขย้อนหลัง แม้ภายหลังข้อมูลภายนอกจะเปลี่ยนแปลง

- Projection Read Models (Local Decision Data)
  เพื่อรองรับการทำงานแบบ Event-Driven และลด Coupling ระหว่างบริการ ระบบจะเก็บสำเนาข้อมูลที่จำเป็นต่อการประเมินในรูปแบบ Local Projection เช่น ข้อมูลสรุปของ Rescue Request, ข้อมูลสถานะ Incident ที่เกี่ยวข้อง, ข้อมูลความพร้อมใช้งานและความเชี่ยวชาญของทีม ข้อมูล Projection เหล่านี้ไม่ถือเป็น Master Data ถูกอัปเดตผ่าน Event จากบริการต้นทาง ใช้เพื่อการประมวลผลภายในเท่านั้น

- Model/Rule Version Metadata
  ข้อมูลเวอร์ชันของ rule set หรือ scoring logic (model version) มีความสำคัญต่อการตรวจสอบความถูกต้องย้อนหลัง และรองรับการปรับปรุงกฎในอนาคตโดยไม่กระทบข้อมูลเก่า รองรับการวิเคราะห์เชิงสถิติภายหลัง

8. Linked Data (Reference Only)

- ก่อนการแนะนำ ระบบจะต้องอ้างอิงข้อมูลจากบริการอื่นเพื่อใช้เป็น input ในกระบวนการประเมินและให้คะแนนคำนวณคำแนะนำ
  - incidentId และ สถานะของ Incident จาก Central Incident Schema

    - ข้อมูลจาก RescueRequest Service
    - ข้อมูลการเปลี่ยนแปลงระดับความรุนแรงจาก IncidentTracking Service
    - ข้อมูลความพร้อมใช้งานของทีมจาก Team/Resource Service
- การเชื่อมโยงข้อมูลจะทำผ่าน Event Subscription เพื่อสร้าง Local Projection แทนการเรียกแบบ synchronous runtime dependency

9. Non-Functional Requirements

Scalability

- Stateless Compute
- รองรับ Horizontal Scaling
- รองรับ Serverless Architecture
- รองรับ Burst Traffic ในช่วง Disaster Spike

Reliability & Fault Tolerance

- รองรับ At-Least-Once Event Delivery
- ต้อง Idempotent ต่อ recommendation_id และ request_id
- มี TTL สำหรับ Recommendation เพื่อลดความเสี่ยงจากข้อมูลล้าสมัย
- หาก Projection ล่าช้า ระบบจะลด Confidence Score
- แยกการทำงานเป็น Synchronous API Handler และ Asynchronous Scoring Worker เพื่อป้องกัน Request Timeout และรองรับปริมาณงานที่พุ่งสูง (Spike) ได้ดีขึ้น

Performance

- การสร้าง Recommendation ต้องไม่ Block Dispatcher
- Re-evaluation ทำงานแบบ Asynchronous
- Latency ในการตอบคำสั่ง Recommendation ต้องอยู่ในระดับที่เหมาะสมต่อการตัดสินใจภาคสนาม

Security

- Role-based Authorization (เฉพาะ Dispatcher / Manage Dispatch Service)
- Audit Log สำหรับตรวจสอบย้อนหลัง
- Correlation ID สำหรับ Distributed Tracing
- รองรับ Event Versioning และ Schema Versioning

Consistency Model

- ใช้แนวคิด Eventual Consistency
- การตัดสินใจอิงจาก Local Projection ที่อัปเดตจาก Event
- ไม่ทำ Distributed Transaction ระหว่างบริการ

Sync Contract

Synchronous Function Contract

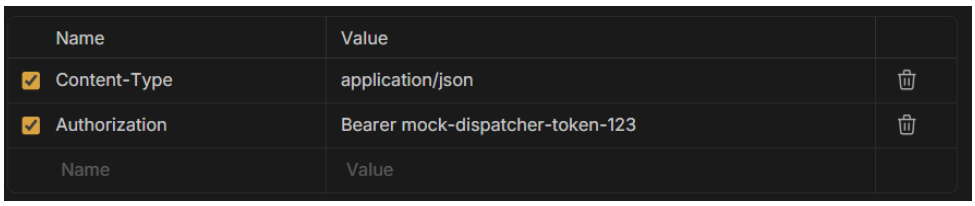
Global Definitions (ข้อกำหนดส่วนกลาง)

- **Authentication:** ทุก API ต้องแนบ Header Authorization: Bearer <dispatcher_token> ซึ่งตอนนี้ได้ทำการ mock <dispatcher_token> ไว้เป็น **mock-dispatcher-token-123**

  ตัวอย่างการใช้งาน Function ในตอนนี้ จึงต้อง แนบ Request Header ดังนี้

  Authorization: Bearer mock-dispatcher-token-123

  หรือตัวอย่างตามภาพนี้



	Name	Value	
☑	Content-Type	application/json	🗑
☑	Authorization	Bearer mock-dispatcher-token-123	🗑
	Name	Value	

- **Traceability:** ทุก API จะคืนค่า trace_id ใน Response เสมอ เพื่อใช้อ้างอิงในการตรวจสอบ Log ในระบบ AWS CloudWatch มีรายละเอียดเพิ่มเติม ดังนี้
  - Unique identifier for each API request
  - Derived from API Gateway requestContext.requestId
  - ใช้สำหรับ
    - tracing logs
    - debugging
    - monitoring distributed systems

- **Error Format:** ทุก Error จะอยู่ในโครงสร้าง ตามตัวอย่างนี้

```
{
  "error": {
    "code": "IDEMPOTENCY_KEY_REUSE_WITH_DIFFERENT_PAYLOAD",
    "message": "Same Idempotency-Key used with different request body",
    "trace_id": "aaUrTgkGIAMEZiw="
    "details": [
      {
        "field": "request_id",
        "issue": "missing"
      }
    ]
  }
}
```

API Contract #1: Generate Recommendation

ข้อมูลทั่วไป

- **Name:** Generate recommendation for rescue request
- **Method:** POST
- **Path:** /v1/recommendations
- **Type:** Synchronous

คำอธิบาย

Endpoint นี้ใช้สำหรับให้ Dispatcher ส่งคำขอเพื่อให้ระบบเริ่มวิเคราะห์และสร้างคำแนะนำทีมกู้ภัยที่เหมาะสมสำหรับ rescue request หนึ่งรายการ

Request

Headers

- Authorization: Bearer <dispatcher_token>
- Content-Type: application/json
- Idempotency-Key: <UUID> (Required)

Body:

```
{
  "request_id": "REQ-8812-9904",
  "force_reevaluate": false
}
```

Validation Rules

- request_id ต้องเป็นค่าที่ถูกต้อง มีอยู่ในข้อมูลของ RescueRequest Service
- หากมี ACTIVE recommendation และ force_reevaluate = false return **409 CONFLICT**
- Idempotency-Key ต้องกำหนดมาใน Header Request
  - หากไม่มี return **400 Bad Request**
  - หากไม่ใช่ UUID (ผิด format) return **400 Bad Request**

Response

Success: 202 Accepted

```
{
  "recommendation_id": "82270dcf-41c2-49ce-b200-1d1b83603c68",
  "request_id": "REQ-8812-9901",
  "incident_id": "a5f61e23-2bb8-4bda-90eb-a352537b81ee",
  "recommendation_status": "PENDING",
  "created_at": "2025-06-01T10:27:027Z"
}
```

**กรณีที่ 202 Accepted คือ Key ไม่ซ้ำ and body ไม่ซ้ำ จะสร้างใหม่

Success: 200 Success


```
{
  "recommendation_id": "82270dcf-41c2-49ce-b200-1d1b83603c68",
  "request_id": "REQ-8812-9901",
  "incident_id": "a5f61e23-2bb8-4bda-90eb-a352537b81ee",
  "recommendation_status": "GENERATED",
  "created_at": "2025-06-01T10:27:027Z"
}
```


**กรณีที่ 200 Success คือ Key ซ้ำ and body ซ้ำ จะไม่สร้างใหม่และ return อันเดิม
**กรณีที่ 200 Success คือ Key ไม่ซ้ำ and body ซ้ำ จะไม่สร้างใหม่และ return อันเดิม

Error: 409 IDEMPOTENCY_CONFLICT


```
{
  "error": {
    "code": "IDEMPOTENCY_KEY_REUSE_WITH_DIFFERENT_PAYLOAD",
    "message": "Same Idempotency-Key used with different request body",
    "trace_id": "aaUrTgkGIAMEZiw="
  }
}
```


**กรณีที่ 409 Conflict คือ Key ซ้ำ and body ไม่ซ้ำ จะไม่สร้างใหม่และ return 409

Error: 409 CONFLICT


```
{
  "error": {
    "code": "ACTIVE_RECOMMENDATION_EXISTS",
    "message": "An active recommendation already exists. Set 'force_reevaluate' to true to override",
    "trace_id": "aaUrTgkGIAMEZiw="
  }
}
```


Header Error: 400 Bad Request


```
{
  "error": {
    "code": "VALIDATION_HEADER_ERROR",
    "message": "Idempotency-Key is required",
    "trace_id": "aaUrTgkGIAMEZiw=",
    "details": [
      {
        "field": "request_id",
        "issue": "missing"
      }
    ]
  }
```

}

Header Error: 400 Bad Request

{
  "error": {
    "code": "VALIDATION_HEADER_ERROR",
    "message": "Invalid idempotency_key format - Idempotency-Key must be a valid UUID",
    "trace_id": "aaUrTgkGIAMEZiw=",
    "details": [
      {
        "field": "request_id",
        "issue": "invalid_format"
      }
    ]
  }
}

Error: 400 Bad Request

{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "request_id is required",
    "trace_id": "aaUrTgkGIAMEZiw=",
    "details": [
      {
        "field": "request_id",
        "issue": "missing"
      }
    ]
  }
}

Error: 404 Not Found

{
  "error": {
    "code": "REFERENCE_NOT_FOUND",
    "message": "request_id not found",
    "trace_id": "aaUrTgkGIAMEZiw="
  }
}

Error: 500 Internal Server Error

{
  "error": {
    "code": "INTERNAL_SERVER_ERROR",

"message": "Fail to create recommendation",
"trace_id": "aaUrTgkGIAMEZiw="
}
}

Dependency / Reliability

- ไม่เรียก service อื่นแบบ synchronous
- ใช้ Local Projection
- Idempotent (ผ่าน Idempotency-Key)
    - If Key ซ้ำ + body เหมือนเดิม return response เดิม return **200 Success**
    - If Key ไม่ซ้ำ + body เหมือนเดิม return response เดิม return **200 Success** (จริงๆ ไม่ควร)
    - If Key ซ้ำ + body ต่าง return **409 IDEMPOTENCY_CONFLICT**
    - If Key ไม่ซ้ำ + body ต่าง สร้างใหม่ return **201 Created**
- บริการต้องสามารถสร้าง Recommendation ได้ภายในระยะเวลาไม่เกิน 2 วินาที ภายใต้ภาระงานปกติ (Normal Operating Load)
- Stateless compute

ข้อมูลอื่นที่สำคัญเช่นกัน

ACTIVE recommendation หมายถึง PENDING, CALCULATING, GENERATED (หากทำ force_reevaluate ตัวเก่าจะเปลี่ยนสถานะเป็น SUPERSEDED ซึ่งถือเป็น INACTIVE)

API Contract #2: Get Recommendation Detail

ข้อมูลทั่วไป

- **Name:** Get Recommendation Detail
- **Method:** GET
- **Path:** /v1/recommendations/{request_id}
- **Type:** Synchronous

คำอธิบาย

ใช้ดึงรายละเอียด Recommendation ด้วย request_id

เนื่องจากระบบรองรับการ force_reevaluate ทำให้ 1 request_id อาจมีหลาย Recommendation ดังนั้น Endpoint นี้จะดึงเอา Recommendation เวอร์ชันล่าสุด (เรียงตาม created_at) มาแสดงผลเสมอ

Request

Path Param

- request_id (String, required)

Headers

- Accept: application/json

- Authorization: Bearer <token>

Validation Rules

- request_id ต้องเป็นค่าที่ถูกต้อง มีอยู่ในข้อมูลของ RescueRequest Service

Response

Success: 200 Success


```
{
  "recommendation_id": "82270dcf-41c2-49ce-b200-1d1b83603c68",
  "request_id": "REQ-8812-9904",
  "recommendation_status": "GENERATED",
  "confidence_score": 4.0,
  "ranked_teams": [
    {
      "team_id": "TEAM-01",
      "rank": 1,
      "total_score": 87.5,
      "score_breakdown": {
        "specialization_score": 30.0,
        "distance_score": 25.0,
        "availability_score": 20.0,
        "severity_weight": 12.5
      },
      "explanation": "Team specialization matches flood response"
    }
  ],
  "model_version": "v1.2.0",
  "evaluated_at": "2025-06-01T10:28:047Z",
  "created_at": "2025-06-01T10:27:027Z"
}
```


Error: 404 NOT_FOUND


```
{
  "error": {
    "code": "REFERENCE_NOT_FOUND",
    "message": "recommendation not found for request_id",
    "trace_id": "aaUrTgkGIAMEZiw="
  }
}
```


Error: 500 Internal Error


```
{
```

```
  "error": {
    "code": "INTERNAL_SERVER_ERROR",
    "message": "Fail to get recommendation detail",
    "trace_id": "aaUrTgkGIAMEZiw="
  }
}
```


Dependency / Reliability

- ไม่เรียก service อื่น
- Idempotent: Yes (read-only operation)

API Contract #3: Delete Recommendation

ข้อมูลทั่วไป

- **Name:** Delete Recommendation
- **Method:** DELETE
- **Path:** /v1/recommendations/{recommendation_id}
- **Type:** Synchronous

คำอธิบาย

Endpoint นี้ใช้สำหรับ ลบ Recommendation ออกจากระบบ โดยอ้างอิงจาก recommendation_id

การลบนี้เป็น Idempotent Operation
หมายความว่า

- หาก recommendation มีอยู่ ระบบจะลบ
- หาก recommendation ไม่มีอยู่ ระบบจะก็จะถือว่าถูกลบไปแล้ว

ทั้งสองกรณีจะ return **204 No Content**

Request

Path Param

- recommendation_id (UUID, required)

Headers

- Accept: application/json
- Authorization: Bearer <dispatcher_token>

Validation Rules

- recommendation_id ต้องเป็นค่า UUID ที่ถูกต้อง มีอยู่ในข้อมูลของ IncidentTracking Service
  - หากไม่มี recommendation_id return **400 Bad Request**
  - หาก format UUID ไม่ถูกต้อง return **400 Bad Request**

Response

Success: 204 No Content

(no body)

Error: 400 Bad Request


```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "recommendation_id is required",
    "trace_id": "aaUrTgkGIAMEZiw=",
    "details": [
      {
        "field": "recommendation_id",
        "issue": "missing"
      }
    ]
  }
}
```


Error: 400 Bad Request


```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "recommendation_id must be a valid UUID",
    "trace_id": "aaUrTgkGIAMEZiw=",
    "details": [
      {
        "field": "recommendation_id",
        "issue": "invalid_format"
      }
    ]
  }
}
```


Error: 500 Internal Server Error


```
{
  "error": {
    "code": "INTERNAL_SERVER_ERROR",
    "message": "Failed to delete recommendation",
    "trace_id": "aaUrTgkGIAMEZiw="
  }
}
```

Dependency / Reliability

- ไม่เรียก service อื่นแบบ synchronous
- ใช้ DynamoDB เป็น data store
- ระบบต้องสามารถดำเนินการลบได้ภายใน 1 วินาที ภายใต้ภาระงานปกติ (Normal Operating Load)
- Stateless compute
- Idempotent: Yes
    - If recommendation exists ⟶ delete ⟶ return 204
    - If recommendation does not exist ⟶ treat as already deleted ⟶ return 204

Future plan

- ระบบลักษณะ Disaster Response ควรมี Audit Trail (หลักฐานย้อนหลัง) โค้ดปัจจุบันคือ **Hard Delete** (ลบข้อมูลหายไปจากตารางแบบถาวร) อนาคต อาจจะ**เปลี่ยนเป็น Soft Delete** (จากการทำ table.delete_item เป็น table.update_item เพื่อเปลี่ยน recommendation_status ให้กลายเป็น DELETED หรือ ARCHIVED แทน)

API Contract #4: Update Recommendation Status

ข้อมูลทั่วไป

- **Name:** Update recommendation for rescue request
- **Method:** PATCH
- **Path:** /v1/recommendations/{recommendation_id}/status
- **Type:** Synchronous

คำอธิบาย

Endpoint นี้ใช้สำหรับให้ Dispatcher อัปเดต Status หรือเลือกทีมจาก Recommendation

Recommendation Lifecycle

States:

- PENDING (INACTIVE)
- CALCULATING (INACTIVE)
- GENERATED (ACTIVE)
- FAILED (INACTIVE)
- ACCEPTED (INACTIVE)
- REJECTED (INACTIVE)
- EXPIRED (INACTIVE)

Rules:

- Only one ACTIVE recommendation per request

Request

Headers

- Authorization: Bearer <dispatcher_token>
- Content-Type: application/json

Body:

```
{
  "recommendation_status": "ACCEPTED",
  "selected_team_id": "TEAM-F01",
  "reason": "(Optional - มักใช้ตอน REJECTED)"
}
```

Validation Rules

- selected_team_id ต้องอยู่ใน ranked list
- ต้องส่ง recommendation_status มาและต้องเป็นค่าที่กำหนด
- recommendation_status ต้องเป็น GENERATED
    - ถ้า status = PENDING / CALCULATING / FAILED / ACCEPTED / REJECTED / EXPIRED return **409 INVALID_STATE**

Response

Success: 200 OK

```
{
  "recommendation_id": "82270dcf-41c2-49ce-b200-1d1b83603c68",
  "recommendation_status": "ACCEPTED",
  "selected_team_id": "TEAM-W01",
  "accepted_at": "2025-06-01T10:27:02Z"
}
```

Error: 400 Bad Request

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "recommendation_id is required",
    "trace_id": "aaUrTgkGIAMEZiw=",
    "details": [
      {
        "field": "recommendation_id",
        "issue": "missing"
      }
    ]
  }
}
```

Error: 400 Bad Request

```
{
```

```
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "recommendation_id must be a valid UUID",
    "trace_id": "aaUrTgkGIAMEZiw=",
    "details": [
      {
        "field": "recommendation_id",
        "issue": "invalid_format"
      }
    ]
  }
}
```


Error: 400 Bad Request


```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "status must be one of ACCEPTED, REJECTED",
    "trace_id": "aaUrTgkGIAMEZiw="
  }
}
```


Error: 404 NOT_FOUND **# ******กำลังพัฒนา**********


```
{
  "error": {
    "code": "REFERENCE_NOT_FOUND",
    "message": "recommendation not found for recommendation_id",
    "trace_id": "aaUrTgkGIAMEZiw="
  }
}
```


Error: 409 Conflict


```
{
  "error": {
    "code": "STATE_CONFLICT",
    "message": "Recommendation must be in GENERATED status to be updated",
    "trace_id": "aaUrTgkGIAMEZiw="
  }
}
```


Error: 500 Internal Server Error


```
{
  "error": {
    "code": "INTERNAL_SERVER_ERROR",
```

```
    "message": "Fail to update recommendation status",
    "trace_id": "aaUrTgkGIAMEZiw="
  }
}
```

Async Contract

Asynchronous Function Contract

Message Contract #1: Process Recommendation Task

ข้อมูลทั่วไป

- **Message Name:** RecommendationProcessingRequested
- **Interaction Style:** Fire-and-Forget / Async Worker Task
- **Producer:** RecommendRescueTeam Service (API Handler)
- **Consumer:** RecommendRescueTeam Service (Scoring Worker)
- **Channel/Queue:** rescue.recommendation.tasks.v1 (หรือ AWS Lambda Async Internal Queue)
- **Version:** v1

คำอธิบาย

Message นี้ ส่งคำสั่งการคำนวณและประเมินคะแนนทีมกู้ภัยแบบ Asynchronous ไปยัง Scoring Worker หลังจากที่ API Handler ตอบกลับ HTTP 202 ให้ผู้ใช้งานแล้ว เพื่อป้องกันปัญหา Timeout ในการคำนวณที่ซับซ้อน

ลำดับการทำงาน

- เริ่มจาก **API Contract #1: Generate Recommendation (**POST /v1/recommendations) เพื่อสร้าง Recommendation โดยที่จะสร้างเพื่อรับทราบเท่านั้น โดยจะ return 202 Accept ทันที พร้อมกับ recommendation_id และข้อมูลอื่นๆ

- ซึ่งใน **API Contract #1: Generate Recommendation** นั้นก่อนที่จะ return 202 Accept ทันที จะมีการสร้าง Request (Event Payload) และ ส่ง Asynchronous Invoke ไปยัง Consumer (ในตอนนี้ใช้เป็น Producer เป็น Lambda ตัวนึง (API Handler) ที่เรียกใช้ Consumer เป็น Lambda อีกตัวนึง (Scoring Worker))

- ต่อมา Consumer ที่เป็น Lambda อีกตัวนึง (Scoring Worker)) ก็จะเริ่มทำงาน โดยการทำงานมีรายละเอียด คร่าวๆ ดังนี้

1. **Acknowledge:** อัปเดต recommendation_status ใน DynamoDB จาก PENDING เป็น CALCULATING ทันที
2. **Fetch Data:** ดึงข้อมูล Request ปัจจุบัน และข้อมูล Candidate Rescue Teams
3. **Compute:** ประเมินคะแนนแต่ละทีมตาม Rule-based Scoring Engine (Specialization, Distance, Availability, Severity)
4. **Sort & Rank:** จัดลำดับทีมจากคะแนนรวมที่ได้
5. **Complete:** อัปเดตข้อมูล ranked_teams, confidence_score ลง Database และเปลี่ยนสถานะจาก CALCULATING เป็น GENERATED
6. **Error Handling:** หากเกิด Exception หรือคำนวณล้มเหลวกลางคัน ระบบจะดักจับ (Catch) และเปลี่ยนสถานะเป็น FAILED เพื่อป้องกันไม่ให้งานค้าง (Deadlock)

Request (Event Payload)

Message Headers (required unless stated)

- message_type: RecommendationProcessingRequested
- message_id: UUID (ใช้เป็น unique identifier ของ event นี้)
- reply_to: dispatch.team.results.v1

- sent_at: ISO-8601 datetime
- trace_id: string (รหัสสำหรับการทำ Distributed Tracing)

Message Body


```
{
  "recommendation_id": "219d078a-5c63-4387-a3bf-6f61e2c9c6eb",
  "request_id": "REQ-8812-9901",
  "incident_id": "019C774D-1AC5-75BB-AE95-5CD4AEB8925B",
}
```


Field Definition:

Field	Type	Required	Description
recommendation_id	UUID	Y	รหัสของ Recommendation ที่สร้างเตรียมไว้ในสถานะ PENDING
request_id	string	Y	รหัสคำร้องขอความช่วยเหลือจาก RescueRequest Service
incident_id	UUID	Y	รหัสเหตุการณ์ภัยพิบัติที่เกี่ยวข้องจาก Central Incident Schema

Validation Rules

1. recommendation_id ต้องเป็นค่า UUID format ที่ถูกต้องและมีอยู่ใน Database
2. สถานะเริ่มต้นของ recommendation ใน Database ต้องเป็น PENDING เท่านั้น
3. request_id ต้องเป็นค่าที่ถูกต้องและมีอยู่ใน RescueRequest Service
4. incident_id ต้องเป็นค่าที่ถูกต้องและมีอยู่ใน Central Incident Schema

Response

เนื่องจาก Interaction Style ในบริบทนี้เป็นแบบ Fire-and-Forget ภายใน Service เดียวกัน ตัว Worker นำผลลัพธ์บันทึกลง Database โดยตรง แทนที่จะตอบกลับเป็น Message ผ่าน Broker ผลลัพธ์จะถูกกำหนดเป็น State Update ดังนี้:

Success Data Update (Database Document)


```
{
  "recommendation_id": "219d078a-5c63-4387-a3bf-6f61e2c9c6eb",
  "request_id": "REQ-8812-9901",
  "recommendation_status": "GENERATED",
```

```
  "confidence_score": 4,
  "ranked_teams": [
    {
      "team_id": "TEAM-W01",
      "rank": 1,
      "total_score": 79.72,
      "score_breakdown": {
        "specialization_score": 30.0,
        "distance_score": 9.72,
        "availability_score": 20.0,
        "severity_weight": 20.0
      },
      "explanation": "Team specialization matches flood rescue | Distance: 13.5 km"
    },
    {
      "team_id": "TEAM-F01",
      "rank": 2,
      "total_score": 40.0,
      "score_breakdown": {
        "specialization_score": 0.0,
        "distance_score": 0.0,
        "availability_score": 20.0,
        "severity_weight": 20.0
      },
      "explanation": "Distance: 540.8 km"
    }
  ],
  "evaluated_at": "2026-03-21T13:30:001Z"
}
```


Reject/Error Data Update (Database Document)


```
{
  "recommendation_id": "9f5b86ff-a2e7-4cda-b463-b7404ba5a791",
  "recommendation_status": "FAILED",
  "error_code": "DATA_FETCH_FAILED",
  "error_message": "Unable to fetch current location of rescue teams"
}
```


Field Definition (Updated State):

Field	Type	Required	Description
recommendation_id	UUID	Y	รหัสของ Recommendation ที่สร้างเตรียมไว้ในสถานะ PENDING
request_id	string	Y	รหัสคำร้องขอความช่วยเหลือจาก RescueRequest Service
recommendation_status	enum	Y	สถานะหลังการประมวลผล (GENERATED หรือ FAILED)
confidence_score	int	Y (If Success)	ค่าความเชื่อมั่นของผลการจัดอันดับ (เช่น 1 - 5)
ranked_teams	array	Y (If Success)	รายชื่อทีมกู้ภัยที่ผ่านการประเมินและจัดอันดับแล้ว
model_version	string	Y (If Success)	เวอร์ชันโมเดล
evaluated_at	datetime	Y (If Success)	เวลาปรับปรุงล่าสุด
error_code	string	Y (If Failed)	รหัสข้อผิดพลาดเพื่อใช้ในการตรวจสอบ (Machine-readable)
error_message	string	Y (If Failed)	รายละเอียดข้อผิดพลาดเพื่อใช้ในการ debug (Human-readable)

Consumer Behavior & Validation Rules

- เมื่อ Consumer ได้รับข้อความ จะต้องอัปเดตสถานะ (recommendation_status) ในฐานข้อมูลเป็น CALCULATING ก่อนเริ่มประมวลผล
- มีการดึงข้อมูล Request ปัจจุบัน และข้อมูล Candidate Rescue Teams
- ถ้าสถานะเป็น GENERATED ต้องมีข้อมูล ranked_teams เสมอ
- ถ้าเกิด Exception ระหว่างการดึงข้อมูลจากภายนอกหรือการคำนวณ Consumer ต้องดักจับ Error และอัปเดตสถานะเป็น FAILED พร้อมระบุ error_code เสมอ เพื่อป้องกันไม่ให้งานค้าง (Deadlock)

Reliability & Delivery Guarantees

- Delivery Model: At-least-once
- Consumer ต้องรองรับ duplicate message
- Producer ใช้ Outbox Pattern เพื่อป้องกัน lost message
- Ordering guarantee ภายใน scope ของ recommendation_id

- **Retry Mechanism:** อาศัยกลไก Internal Retry ของ AWS Lambda (อ้างอิงตามมาตรฐาน Async Invoke ของ AWS หาก Worker ทำงานล้มเหลว ระบบจะทำการ Retry อัตโนมัติสูงสุด 2 ครั้ง)
- **Idempotency:** การคำนวณเป็นแบบ Idempotent หากได้รับ Event ที่มี recommendation_id เดิมซ้ำ ผลลัพธ์การจัดอันดับและคะแนนที่ได้จะยังคงเหมือนเดิม (อิงจาก Snapshot ของข้อมูล ณ เวลานั้น)
- **Traceability:** ใช้ trace_id ที่ส่งต่อมาจาก API Gateway เพื่อให้สามารถทำ Distributed Tracing ดู Log เชื่อมโยงกันระหว่าง API Handler และ Worker ได้

Architectural Rationale

- Recommendation Service ไม่ต้องรอ Dispatch Service แบบ synchronous
- ลด coupling ระหว่าง domain
- รองรับ scaling แยกอิสระ
- รองรับ retry โดยไม่เกิด side effect (idempotent by message_id)

Service Data

Service Data

หมายเหตุ: บริการนี้ใช้ Amazon DynamoDB (NoSQL Database) ในการจัดเก็บข้อมูล ดังนั้นโครงสร้างข้อมูลจึงออกแบบในลักษณะ Document-based (Single Table Design) โดยมีการฝังข้อมูล (Embedded Data) เข้าไปใน Document หลัก เพื่อลดความซับซ้อนในการ Query และเพิ่มประสิทธิภาพในการอ่าน/เขียน

1) Recommendation Master Data (Owned by this service)

Field Name	Type	Required	Description	Example
recommendation_id	uuid	Y (PK)	รหัส recommendation	b94cb0eb-b627-41b9-a009-ed777ab3db38
request_id	string	Y (reference only)	(GSI / Reference) อ้างอิงคำร้องขอความช่วยเหลือจาก RescueRequest Service	REQ-8812-9901
incident_id	uuid	Y (reference only)	(Reference) อ้างอิงเหตุการณ์ภัยพิบัติจาก Incident Service	e7fbc96c-24a6-49e5-95c7-1581f656de4b
recommendation_status	enum	Y	สถานะปัจจุบันของคำแนะนำ	GENERATED
confidence_score	integer (1-5)	N	ค่าความมั่นใจ	4
ranked_teams	array	N	(Embedded Array) รายการทีมกู้ภัยที่ผ่านการจัดอันดับ (ดูโครงสร้างย่อยในข้อ 2)	[{ team_id: "TEAM-W1"...}, { team_id: "TEAM-W2"...}]
model_version	string	N	เวอร์ชันของ Scoring Logic (Model)	v1.2.0
idempotency_key	uuid	Y	(GSI) คีย์เพื่อป้องกันการประมวลผลซ้ำซ้อน	abfbc96c-24a6-49e5-95c7-1581f656de4b
request_hash	string	Y	ค่า Hash ของ Request Body ไว้เช็คว่า Payload เดิมหรือไม่	a94a8fe5ccb1...
selected_team_id	string	N	รหัสทีมที่ Dispatcher ตัดสินใจเลือก (เกิดขึ้นเมื่อสถานะเป็น ACCEPTED)	TEAM-W1
status_reason	string	N	เหตุผลประกอบการเปลี่ยนสถานะ (มักใช้ตอน REJECTED)	ทีมอุปกรณ์ไม่พอ

created_at	datetime	Y	เวลาที่รับ Request เข้าสู่ระบบ (PENDING)	2025-06-01T10:27:023Z
evaluated_at	datetime	N	เวลาที่ Worker ประเมินคะแนนเสร็จสิ้น (GENERATED)	2025-06-01T10:28:024Z
updated_at	datetime	N	เวลาที่มีการเปลี่ยนสถานะล่าสุด (เช่น ACCEPTED หรือ SUPERSEDED)	2025-06-01T10:27:021Z

2) Ranked Team Object (Embedded Model)

โครงสร้างข้อมูลออบเจกต์ที่ฝังอยู่ภายใน Array ranked_teams ของตารางหลัก (ไม่ต้องทำ Foreign Key เนื่องจากอยู่ใน Document เดียวกัน)

Field Name	Type	Required	Description	Example
team_id	string	Y	รหัสทีมกู้ภัยที่ถูกจัดอันดับ	TEAM-W01
rank	integer	Y	ลำดับความเหมาะสม (1 = ดีที่สุด)	1
total_score	float	Y	คะแนนรวมทั้งหมดหลังคำนวณ	92.75
score_breakdown	object	Y	(Nested Object) รายละเอียดที่มาของคะแนนแยกตามหัวข้อ เพื่อรองรับ Explainability	{ "specialization_score": 30.0, "distance_score": 9.72, "availability_score": 20.0, "severity_weight": 20.0 }
explanation	integer	Y	คำอธิบายเหตุผลที่ทีมนี้ได้คะแนนดังกล่าว (Human-readable)	Team specialization matches...

(หมายเหตุ: โครงสร้างของ score_breakdown ประกอบด้วย specialization_score, distance_score, availability_score, และ severity_weight เป็นชนิด float ทั้งหมด)

Enum Definition

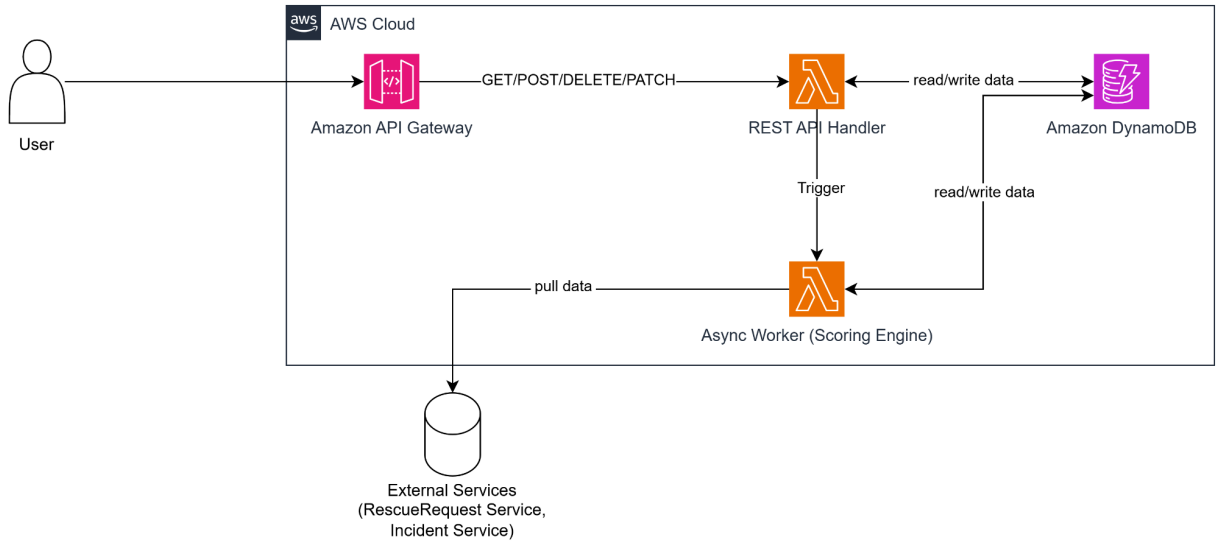
recommendation_status:

- PENDING (รอประมวลผล)

- CALCULATING (กำลังประมวลผล)
- GENERATED (จัดอันดับเสร็จสิ้น)
- ACCEPTED (Dispatcher ยอมรับและเลือกทีม)
- REJECTED (Dispatcher ปฏิเสธคำแนะนำ)
- SUPERSEDED (ถูกแทนที่เนื่องจากมีการ Force Re-evaluate)
- FAILED (เกิดข้อผิดพลาดในการคำนวณ)

Service Architecture

Service Architecture



Components

- **User (Dispatcher / Manage Dispatch Service):** ผู้เรียกใช้งานระบบผ่าน REST API เพื่อขอคำแนะนำและจัดการสถานะคำแนะนำ
- **Amazon API Gateway:** จุดรับคำขอแบบ Synchronous ทั้งหมด (POST, GET, PATCH, DELETE) และจัดการเรื่อง Routing
- **Lambda REST API Handler:** ประมวลผลคำขอเบื้องต้น, ตรวจสอบสิทธิ์ (Authorization), ตรวจสอบ Idempotency, อ่าน/เขียนข้อมูลพื้นฐาน และตอบกลับผู้ใช้ทันที
- **Lambda Async Worker (Scoring Engine):** รับ Event Trigger แบบ Asynchronous เพื่อรันอัลกอริทึมประเมินคะแนน (Scoring Logic) และจัดอันดับทีมกู้ภัยเป็น Background Job
- **Amazon DynamoDB:** ฐานข้อมูล NoSQL หลักของบริการ ทำหน้าที่จัดเก็บ Recommendation Document, Ranked Teams Snapshot, และข้อมูลสถานะ (State)
- **External Services (RescueRequest / Incident Service):** บริการภายนอกที่ระบบต้องดึงข้อมูล (หรืออ้างอิงจาก Local Projection) มาใช้ประกอบเป็นบริบทในการคำนวณคะแนน

Explanation

สถาปัตยกรรมของ RecommendRescueTeam Service ถูกออกแบบมาเพื่อทำหน้าที่เป็น Decision-Support Engine ที่ตอบสนองได้อย่างรวดเร็วและไม่บล็อกการทำงานของผู้ใช้งาน (Non-blocking) โดยแบ่งการทำงานออกเป็น 2 ส่วนหลักคือ Synchronous API และ Asynchronous Processing

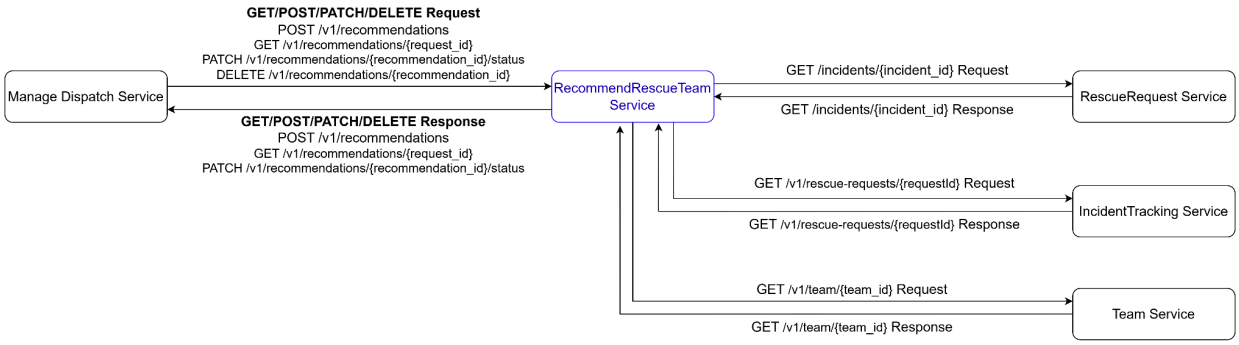
สำหรับการทำงานแบบ **Synchronous** ผู้ใช้งาน (Dispatcher) จะเรียกใช้งานผ่าน Amazon API Gateway เพื่อดึงรายละเอียดคำแนะนำ (GET), อัปเดตสถานะการตัดสินใจ (PATCH), หรือลบข้อมูล (DELETE) คำขอเหล่านี้จะถูกส่งไปยัง **Lambda REST API Handler** ซึ่งจะทำหน้าที่ตรวจสอบความถูกต้อง อ่านหรืออัปเดตข้อมูลใน Amazon DynamoDB และตอบกลับผลลัพธ์ให้ผู้ใช้ทันที รูปแบบนี้ทำให้ระบบมีความคล่องตัวและตอบสนองต่อคำสั่งพื้นฐานได้อย่างรวดเร็ว

สำหรับการสร้างคำแนะนำใหม่ (Generate Recommendation) ซึ่งต้องอาศัยการประมวลผลทางคณิตศาสตร์ (เช่น Haversine Formula) และกฎเกณฑ์ที่ซับซ้อน ระบบจะใช้รูปแบบ **Asynchronous** เมื่อ API Handler ได้รับคำขอ (POST) ระบบจะบันทึกสถานะเริ่มต้นเป็น PENDING ลงใน DynamoDB และตอบกลับผู้ใช้ด้วย HTTP 202 Accepted ทันที จากนั้น API Handler จะทำการ **Asynchronous Invoke (Event Trigger)** ไปยัง **Lambda Async Worker** เพื่อให้รับช่วงต่องาน

ในเบื้องหลัง (Background) ตัว Worker จะดึงข้อมูลจาก External Services นำมาเข้าสมการให้คะแนน จัดอันดับทีม และอัปเดตผลลัพธ์พร้อมเปลี่ยนสถานะเป็น GENERATED กลับลงในฐานข้อมูล สถาปัตยกรรมแบบแยกส่วน (Decoupled Compute) นี้ช่วยป้องกันปัญหา Request Timeout และทำให้ระบบรองรับปริมาณงานที่พุ่งสูง (Burst Traffic) ในช่วงเวลาเกิดภัยพิบัติได้อย่างมีประสิทธิภาพ

Service Interaction

Service Interaction



Upstream Services (บริการต้นทางที่เรียกใช้งาน RecommendRescueTeam Service)

Dispatcher Dashboard / Manage Dispatch Service

- เรียกใช้งาน POST /v1/recommendations แบบ synchronous เพื่อส่งคำขอให้ระบบเริ่มวิเคราะห์และสร้างคำแนะนำทีมกู้ภัย โดยระบบจะตอบรับทันที (HTTP 202 Accepted) และนำคำขอไปประมวลผลต่อแบบ asynchronous เป็น Background Job
- เรียกใช้งาน GET /v1/recommendations/{request_id} แบบ synchronous เพื่อดึงข้อมูลผลลัพธ์การจัดอันดับทีมกู้ภัยเวอร์ชันล่าสุด พร้อมรายละเอียดคะแนน (Score Breakdown) และคำอธิบาย (Explanation) เพื่อสนับสนุนการตัดสินใจของ Dispatcher
- เรียกใช้งาน PATCH /v1/recommendations/{recommendation_id}/status แบบ synchronous เพื่อส่งผลการตัดสินใจของมนุษย์กลับเข้าระบบ (เช่น ACCEPTED พร้อมเลือกทีม หรือ REJECTED พร้อมระบุเหตุผล)
- เรียกใช้งาน DELETE /v1/recommendations/{recommendation_id} แบบ synchronous เพื่อลบหรือยกเลิกคำแนะนำในกรณีที่ข้อมูลไม่จำเป็นต้องใช้งานแล้ว

Downstream Services (บริการปลายทางที่ RecommendRescueTeam Service เรียกหรือดึงข้อมูลมาใช้)

- **RescueRequest Service**
  - ถูกเรียกใช้งานแบบ synchronous (หรืออ้างอิงผ่าน Local Projection) เพื่อดึงข้อมูลรายละเอียดของคำร้องขอความช่วยเหลือ เช่น ประเภทเหตุการณ์ (Request Type), พิกัดสถานที่ (Location), และระดับความรุนแรง (Priority Level) เพื่อนำมาเป็น Input หลักในการคำนวณ
- **Incident Service**
  - ถูกเรียกใช้งานเพื่ออ้างอิงข้อมูลสถานะและบริบทภาพรวมของเหตุการณ์ภัยพิบัติใหญ่ (Incident) ที่คำร้องขอนั้นสังกัดอยู่ เพื่อใช้ปรับน้ำหนักความรุนแรง (Severity Weight)
- **Team / Resource Service (บริการจัดการทีมกู้ภัย)**
  - อ้างอิงข้อมูลเพื่อดึงรายการทีมกู้ภัยทั้งหมด (Candidate Teams) โดยดึงข้อมูลสถานะความพร้อมใช้งาน (Availability), ความเชี่ยวชาญ (Specialties), และพิกัดปัจจุบัน (Current Location) เพื่อนำเข้าสู่สมการ Scoring Engine สำหรับคำนวณระยะทางและความเหมาะสม